

SOCIAL NETWORKS

NPTEL Online Course • Prof. Sudarshan Iyengar, IIT Ropar

Week 1 — Question Paper (Comprehension-Based MCQ / MSQ)

Comprehension 1 — The First Week at Nalanda: Why Gossip Always Reaches Everyone

Case Study

Thirty-six first-year students join a fresh batch at Nalanda Institute. On Day 1 almost none of them know one another. Yet by the end of Day 2, a single announcement — “the orientation trek has been postponed” — has reached every single student, even though each student has personally chatted with only three or four others.

A teaching assistant, Meera, sets out to explain this. She models the batch as a graph: every student is a node, and a mutual acquaintance between two students is an undirected edge. A piece of news, she reasons, can travel only along edges — from a student to their acquaintances, then onward to the acquaintances’ acquaintances.

To test her intuition she runs a simulation. For each of the 36 students she picks 3 other students uniformly at random and declares them acquaintances. She draws the resulting graph and checks whether it is connected — whether news starting from any one student could reach all the rest. She repeats the whole experiment 1000 times. Almost every single time, the graph turns out to be connected.

Puzzled that so few acquaintances suffice, Meera considers the opposite scenario: could the batch instead split into two groups of 18 students each, with acquaintances only inside each group and not a single one crossing between them? She also sketches, for a small 6-member project team, the “everyone-knows-everyone” graph. Finally she wonders just how many distinct acquaintance graphs are even possible on a batch this size — a number, she suspects, large enough to dwarf the count of atoms in the universe.

Questions 1–10

Q1. [MCQ] Meera claims the postponement notice can reach all 36 students if and only if a certain property holds for her acquaintance graph. Which property?

- A. Every student is acquainted with every other student (the graph is complete).
- B. The graph is connected — there is a path between every pair of students.
- C. Every student has at least three acquaintances.
- D. The graph contains no cycles (it is a tree).

Answer: B

Reference: Lecture 1 – Introduction; Lecture 2 – Answer to the puzzle (≈06:00–07:00)

Solution: News travels only along edges, so it reaches every node from any starting node exactly when the graph is connected (a path exists between every pair). Completeness (A) and a minimum degree of three (C) look sufficient but are not necessary — a connected graph can contain low-degree nodes and need not be complete. A tree (D) is just one special connected graph, not a requirement.

Q2. [MCQ] What is the maximum possible number of acquaintance edges in the 36-student batch?

- A. $36 \times 36 = 1296$
- B. $36 \times 35 = 1260$
- C. $36 \times 35 / 2 = 630$
- D. $35 \times 34 / 2 = 595$

Answer: C

Reference: Lecture 5 – Introduction to NetworkX-1 (≈05:11)

Solution: The maximum number of edges on n nodes is $n(n-1)/2$, counting each unordered pair once. For $n = 36$ this is $36 \cdot 35 / 2 = 630$. Option B double-counts ordered pairs, A includes impossible self-pairs, and D mistakenly uses 35 nodes.

Q3. [MCQ] In the “two groups of 18” scenario, how many distinct cross-group acquaintances (edges with one endpoint in each group) are even possible — and why does this make a perfect split improbable?

- A. 18 possible cross edges; improbable because at least 18 must be absent.
- B. 36 possible cross edges; improbable because the groups are equal in size.
- C. 324 possible cross edges; improbable because even a single one of them, if present, would merge the two groups into one connected component.
- D. 153 possible cross edges; improbable because 153 is the number of edges inside one group.

Answer: C

Reference: Lecture 2 – Answer to the puzzle (≈02:34–03:23)

Solution: Cross edges = $18 \times 18 = 324$, since each of the 18 students on one side can pair with each of the 18 on the other. For a clean bifurcation all 324 must be absent; even one present links the halves. With so many independent chances, the probability that none occurs is vanishingly small — which is why the random experiment almost always yields a connected graph. (The 153 in D is $C(18,2)$, the count of within-group pairs, not cross pairs.)

Q4. [MSQ] Meera’s simulation gives a connected graph almost every time. Which of the following are correct conclusions? (Select all that apply.)

- I. A connected graph on 36 nodes can exist with far fewer than 630 edges.
 - II. If even one student ends up with zero acquaintances, the graph cannot be connected.
 - III. “Connected” guarantees a direct edge between every pair of students.
 - IV. Once the graph is connected, news starting at any single student can in principle reach all others.
- A. I, II, IV
 - B. I, III, IV
 - C. II, III
 - D. I, II, III, IV

Answer: A (statements I, II, IV)

Reference: Lecture 1 – Introduction; Lecture 2 – Answer to the puzzle

Solution: I is true — the screencast’s “3 acquaintances each” experiment connects all 36 students with only about 50 edges, far fewer than the 630 maximum, so a connected graph certainly does not need every possible edge. II is true — an isolated node has no path to anyone, breaking connectivity. IV is true — connectivity means a path from any node to every other. III is false — connectivity requires a path, not necessarily a direct edge; requiring a direct edge between every pair would describe a complete graph.

Q5. [MCQ] For the 6-member project team modelled as a complete graph T , Meera computes $A = T.order() / T.size()$. What is A ?

- A. $1/3$
- B. $2/5$
- C. $1/2$
- D. $1/5$

Answer: B

Reference: Lecture 5 – Introduction to NetworkX-1 (≈06:28–06:42)

Solution: `order()` is the number of nodes = 6, and `size()` is the number of edges = $6 \cdot 5 / 2 = 15$. So $A = 6/15 = 2/5$. In general, for a complete graph on n nodes the ratio is $n / [n(n-1)/2] = 2/(n-1)$; for $n = 6$ that is $2/5$.

Q6. [MCQ] Meera wants the total number of distinct acquaintance graphs possible on the 36 labelled students — counting every possible set of edges, from the empty graph all the way to the complete graph. This number equals:

- A. 2^{630}
- B. 2^{36}
- C. 36^2
- D. $36!$

Answer: A

Reference: Lecture 7 – Social Networks: The Challenge (≈01:39)

Solution: Each of the 630 possible edges is independently either present or absent, giving 2 choices per edge, hence 2^{630} graphs in total — exactly $2^{(n(n-1)/2)}$. This dwarfs the estimated number of atoms in the universe ($\approx 2^{200+}$), which is why brute-force enumeration of social networks is hopeless and is the central “challenge” of the subject.

Q7. [MCQ] Suppose after the experiment one student, Ravi, is picked by no one and shares no acquaintance with anyone (he forms an isolated node). What does Meera’s model predict about the postponement notice?

- A. The notice still reaches Ravi because the rest of the class is connected.
- B. The notice reaches everyone except Ravi, and the graph is not connected.
- C. The notice reaches no one, because one isolated node disconnects the graph for everybody.
- D. The notice reaches Ravi first and then spreads outward from him.

Answer: B

Reference: Lecture 1 – Introduction; Lecture 2 – Answer to the puzzle

Solution: An isolated node has no path to any other node, so the graph is disconnected and Ravi never hears the news. The remaining 35 students, if connected among themselves, still pass the news around. A disconnection isolates only the separated part — it does not stop spread within the connected part (so C is wrong).

Q8. [MCQ] To check her hand calculation, Meera runs the following. What does it print?

```
import networkx as nx
T = nx.complete_graph(6)
print(T.size(), T.order())
```

- A. 6 15
- B. 15 6
- C. 30 6
- D. 15 30

Answer: B

Reference: Lecture 5 – Introduction to NetworkX-1 (≈06:28–06:42)

Solution: `size()` returns the number of edges (15 for the complete graph on 6 nodes) and `order()` returns the number of nodes (6). Because the print order is `size()` then `order()`, the output is 15 6. Contrast this with Q5, where the ratio `order/size` = 6/15.

Q9. [MCQ] Instead of the “3 acquaintances each” rule, Meera models acquaintance formation as a coin toss for every possible pair. Which statement best describes the graph G she builds below?

```
G = nx.gnp_random_graph(36, 0.1)
```

- A. A graph on 36 nodes in which exactly 10% of all 630 possible edges are guaranteed to be present.
- B. A graph on 36 nodes where each of the 630 possible edges is independently included with probability 0.1.
- C. A connected graph on 36 nodes in which every node has exactly 0.1×35 acquaintances.
- D. A graph on 10 nodes with edges added with probability 36.

Answer: B

Reference: Lecture 5 – Introduction to NetworkX-1 (≈08:18–09:50)

Solution: `nx.gnp_random_graph(n, p)` creates `n` nodes and, for every possible pair, includes that edge independently with probability `p` — a coin toss per edge. The actual edge count is random and only averages $p \times 630$; it is not fixed at exactly 63, and node degrees are not fixed either (so A and C, which assert exact counts, are wrong). D simply swaps the two arguments.

Q10. [MSQ] Reflecting on the course’s central “challenge,” which statements are correct? (Select all that apply.)

- I. The number of possible graphs on n nodes grows as $2^{(n(n-1)/2)}$, i.e. exponentially in the number of possible edges.

- II. For just 50 students the number of possible graphs already exceeds the estimated number of atoms in the universe.
- III. Because the space of graphs is so large, no useful regularities can ever be found in real social networks.
- IV. Despite the enormous space of possibilities, real social networks tend to display nice, predictable emergent properties (such as connectedness from very few links).

- A. I, II, IV
- B. I, II, III
- C. II, IV
- D. I, III, IV

Answer: A (statements I, II, IV)

Reference: Lecture 7 – Social Networks: The Challenge

Solution: I and II restate the combinatorial explosion: $2^{\binom{50}{2}} = 2^{1225}$, far beyond the atoms in the universe ($\approx 2^{200+}$). IV is the course's key motivating insight — structure emerges despite the huge space. III is false and contradicts IV; the entire point is that strong regularities do emerge and can be studied.

Comprehension 2 — Routing for a Logistics Startup: Building a Road Network in Python + NetworkX

Case Study

ShipFast, a logistics startup, asks an intern, Aanya, to prototype a road-network model so the company can compute cheapest routes between cities. Aanya models each city as a node and each road as an undirected, weighted edge whose weight is the travel cost (in rupees) along that road.

She first warms up with plain Python — the random module, lists and dictionaries, and matplotlib — and then builds the graph in NetworkX. For her prototype she uses five cities and the following roads:

Delhi–Jaipur: 300, Jaipur–Mumbai: 1100, Delhi–Ahmedabad: 900, Ahmedabad–Mumbai: 600, Jaipur–Ahmedabad: 650. (There is no direct Delhi–Mumbai road.)

She builds this with `G = nx.Graph()` and repeated `G.add_edge(city1, city2, weight=cost)` calls, then uses NetworkX's connectivity and shortest-path tools — including Dijkstra's algorithm — to answer routing questions, taking care to handle the case where two cities have no connecting route at all.

Questions 1–10

Q1. [MCQ] Aanya keeps a list of candidate road costs and picks one at random for each road, exactly as in the city-network screencast. Which set lists exactly the values `w` can take?

```
import random
costs = [100, 200, 300, 400, 500]
w = random.choice(costs)
```

- A. Any integer between 100 and 500, including 250 or 333
- B. {100, 200, 300, 400, 500}
- C. {200, 300, 400, 500}
- D. Only 100 (the first element of the list)

Answer: B

Reference: Lecture 6 – Introduction to NetworkX-2 (≈10:58, `random.choice` on a list)

Solution: `random.choice(costs)` returns one element chosen uniformly at random from the list, so `w` can be any one of the five values actually stored in `costs`: 100, 200, 300, 400 or 500. It never invents in-between values such as 250 (A), and it is not restricted to the first element (D); each call may return a different stored value.

Q2. [MCQ] Aanya tests two calls. Which statement is correct about the possible values?

```
a = random.randint(2, 5)
b = random.randrange(2, 5)
```

- A. $a \in \{2, 3, 4, 5\}$ and $b \in \{2, 3, 4\}$
- B. $a \in \{2, 3, 4\}$ and $b \in \{2, 3, 4, 5\}$

- C. $a \in \{3, 4, 5\}$ and $b \in \{2, 3, 4, 5\}$
- D. a and b can both take any value in $\{2, 3, 4, 5\}$

Answer: A

Reference: Lecture 3 – Introduction to Python-1 (≈06:39, ≈08:58)

Solution: `random.randint(x, y)` includes both endpoints, giving $\{2, 3, 4, 5\}$. `random.randrange(x, y)` includes the start but excludes the stop, giving $\{2, 3, 4\}$. This start-included / stop-excluded difference is the key contrast between the two functions.

Q3. [MCQ] While experimenting, Aanya runs the snippet below. What is printed?

```
import random
x = []
for i in range(8):
    x.append(random.randint(1, 6))
x.sort()
x.append({"a": 1, "b": 2, "c": 3})
print(len(x))
```

- A. 8
- B. 9
- C. 11
- D. 3

Answer: B

Reference: Lecture 3 – Introduction to Python-1 (≈09:49); Lecture 4 – Python-2

Solution: The loop runs 8 times and appends 8 integers, so after sorting the list has length 8. Appending one dictionary adds a single element — a dictionary inside a list counts as one item regardless of how many keys it holds. The final length is $8 + 1 = 9$.

Q4. [MCQ] Aanya keeps city names in a list and writes the code below. What is the output?

```
cities = ['Delhi', 'Jaipur', 'Mumbai']
try:
    pick = cities[3]
    print("Picked", pick)
except:
    print("No such city")
```

- A. Picked Mumbai
- B. Picked Delhi
- C. No such city
- D. The program crashes with an unhandled error

Answer: C

Reference: Lecture 6 – Introduction to NetworkX-2 (≈35:43, exception handling)

Solution: Valid indices for a 3-element list are 0, 1, 2; `cities[3]` raises an `IndexError`. Control therefore passes to the except block, which prints No such city. Because the error is caught, the program does not crash (so D is wrong).

Q5. [MCQ] Aanya stores a hub's daily truck count in a dictionary and writes the code below. What is the output?

```
trucks = {'Delhi': 25, 'Mumbai': 30}
try:
    print(trucks[0])
except KeyError:
    print("Use the city name, not an index")
```

- A. 25
- B. 30
- C. Use the city name, not an index
- D. 0

Answer: C

Reference: Lecture 4 – Introduction to Python-2 (≈03:32–04:04)

Solution: A dictionary is accessed by key, not by positional index. `trucks[0]` looks for the key 0, which does not exist, so a `KeyError` is raised and the `except` block runs. To read Delhi's value Aanya would write `trucks['Delhi']`.

Q6. [MSQ] Aanya plots costs with matplotlib and numpy. Which statements are correct? (Select all that apply.)

- I. `plt.plot([300, 600, 900])` treats the list as the y-values and auto-assigns `x = 0, 1, 2`.
 - II. After `plt.plot(...)`, `plt.show()` must be called to display the figure.
 - III. If `x = [1, 2, 3]` is an ordinary Python list, `x ** 2` computes `[1, 4, 9]` element-wise.
 - IV. Converting with `numpy.array(x)` first allows element-wise operations like `x ** 2`.
- A. I, II, IV
 - B. I, II, III
 - C. II, III, IV
 - D. I, III

Answer: A (statements I, II, IV)

Reference: Lecture 4 – Introduction to Python-2 (≈12:39 plotting; ≈25:10 numpy)

Solution: I — a single list passed to `plot` is taken as the y-values, with `x` defaulting to indices 0, 1, 2. II — nothing renders until `plt.show()` is called. IV — NumPy arrays support vectorised (element-wise) operations. III is false: applying `**` to a plain Python list raises a `TypeError`, which is exactly why the list must first be converted to a NumPy array.

Q7. [MCQ] Aanya assumes every road takes the same cost in both directions, so a single edge serves both ways. Which constructor matches this modelling choice, and what would the alternative give?

- A. `G = nx.Graph()` makes an undirected graph (one edge serves both directions); `nx.DiGraph()` would make a directed graph.
- B. `G = nx.DiGraph()` makes an undirected graph; `nx.Graph()` would make a directed graph.
- C. `G = nx.Graph()` makes a directed graph; `nx.complete_graph()` would make it undirected.

D. Both `nx.Graph()` and `nx.DiGraph()` create undirected graphs; only the edge weights differ.

Answer: A

Reference: Lecture 6 – Introduction to NetworkX-2 (≈04:00)

Solution: `nx.Graph()` creates an undirected graph, so one edge between two cities represents travel both ways — matching Aanya’s assumption. `nx.DiGraph()` creates a directed graph, where an edge $u \rightarrow v$ does not imply an edge $v \rightarrow u$.

Q8. [MCQ] Using the five-city prototype (Delhi–Jaipur 300, Jaipur–Mumbai 1100, Delhi–Ahmedabad 900, Ahmedabad–Mumbai 600, Jaipur–Ahmedabad 650), Aanya runs the code below. What value is printed?

```
print(nx.dijkstra_path_length(G, 'Delhi', 'Mumbai'))
```

- A. 1100
- B. 1400
- C. 1500
- D. 1550

Answer: B

Reference: Lecture 6 – Introduction to NetworkX-2 (≈30:27–31:31)

Solution: `dijkstra_path_length` returns the minimum total edge weight. The candidate routes are Delhi→Jaipur→Mumbai = 300 + 1100 = 1400; Delhi→Ahmedabad→Mumbai = 900 + 600 = 1500; and Delhi→Jaipur→Ahmedabad→Mumbai = 300 + 650 + 600 = 1550. The cheapest is 1400.

Q9. [MCQ] For the same prototype, Aanya now runs the code below. What is returned, and how does it differ from `dijkstra_path_length`?

```
print(nx.dijkstra_path(G, 'Delhi', 'Mumbai'))
```

- A. It returns the number 1400 — identical to `dijkstra_path_length`.
- B. It returns the list ['Delhi', 'Jaipur', 'Mumbai'] — the sequence of cities on the cheapest route — whereas `dijkstra_path_length` returns the cost 1400.
- C. It returns every possible route between Delhi and Mumbai.
- D. It returns ['Delhi', 'Ahmedabad', 'Mumbai'], the route with the fewest hops.

Answer: B

Reference: Lecture 6 – Introduction to NetworkX-2 (≈31:31); cf. Week-1 sample Q8

Solution: `dijkstra_path` returns the shortest (least-weight) path as a list of nodes — here ['Delhi', 'Jaipur', 'Mumbai'], total cost 1400 — while `dijkstra_path_length` returns only the numeric cost. Dijkstra minimises total weight, not hop count, so even though Delhi→Ahmedabad→Mumbai also has two hops, the cheaper Jaipur route is chosen (D wrong). It returns one optimal path, not all paths (C wrong).

Q10. [MSQ] After deleting some roads, certain cities may become unreachable. Aanya safeguards her code as below. Which statements are correct? (Select all that apply.)

```
try:  
    L = nx.dijkstra_path_length(G, 'Delhi', 'Mumbai')
```

```
except:  
    L = 100000 # stand-in for "infinity"  
print(L)
```

- I. `nx.is_connected(G)` returns a Boolean indicating whether every city is reachable from every other.
 - II. `nx.has_path(G, 'Delhi', 'Mumbai')` returns a Boolean indicating whether at least one route exists between the two cities.
 - III. If Delhi and Mumbai lie in different components, `dijkstra_path_length` raises an exception, which the `try/except` turns into `L = 100000`.
 - IV. The `try/except` is unnecessary, because `dijkstra_path_length` returns 0 when no path exists.
- A. I, II, III
B. I, II, IV
C. II, III, IV
D. I, III

Answer: A (statements I, II, III)

Reference: Lecture 6 – Introduction to NetworkX-2 (≈17:14 `is_connected`; ≈19:23 `has_path`; ≈35:43 exceptions)

Solution: I and II are correct — both functions return Booleans, one for overall connectivity and one for pairwise reachability. III is correct — with no connecting route, `dijkstra_path_length` raises an exception (the screencast shows the “node not reachable” error), and the `except` block substitutes a large number as a stand-in for infinity. IV is false — the function raises an exception rather than returning 0, which is precisely why the `try/except` is needed.